

CSE 3100

Scaffold

Web Programming Laboratory

Project Report

Submitted By:

Rafsan Riasat

Roll: 2207006

Introduction

Modern software projects require more than a task list. Teams need a shared place for planning, discussion, documentation, progress visibility, and integration with their source-code workflow. **Scaffold** addresses these requirements through a workspace-oriented application in which users can create projects, organize tasks, define dependencies, collaborate through structured threads, maintain project wikis, and connect projects to GitHub repositories.

The application follows a Laravel–Inertia–React architecture. Laravel owns routing, validation, authorization, persistence, background jobs, and broadcasting. Inertia.js connects server-side controllers to React pages without requiring a separate REST API for every screen. React provides interactive interfaces such as Kanban and dependency-flow views, while Laravel Reverb and Echo deliver real-time updates.

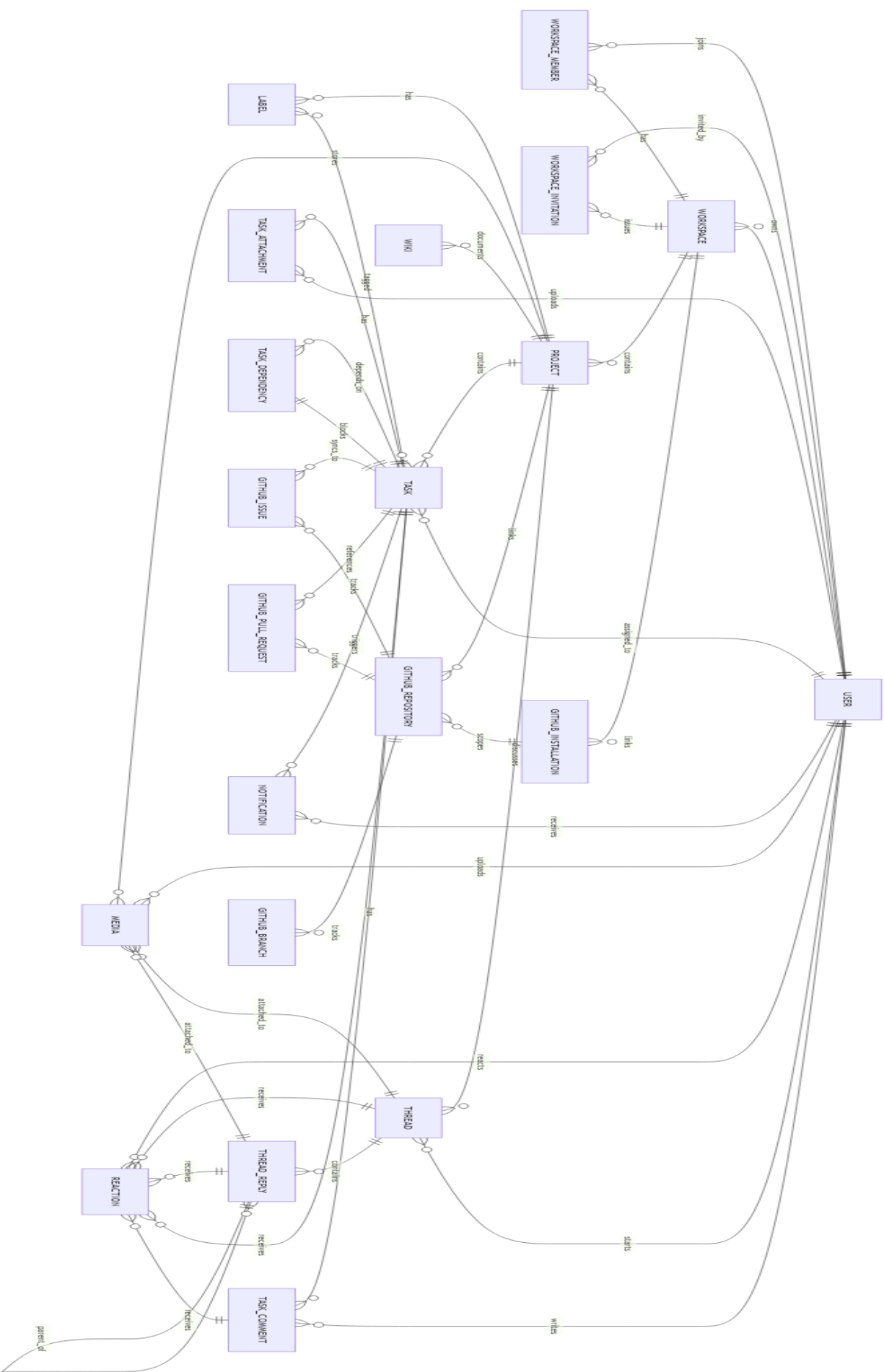
Project objectives

- Provide secure user authentication and account recovery.
- Isolate data within workspaces and projects.
- Support task management, assignment, labeling, dependencies, and status flow.
- Enable threaded technical discussion and reactions.
- Maintain lightweight project documentation through a wiki.
- synchronize suitable task and repository activity with GitHub.
- Update collaborating users in real time.
- Deploy the application publicly using cloud-hosted infrastructure.

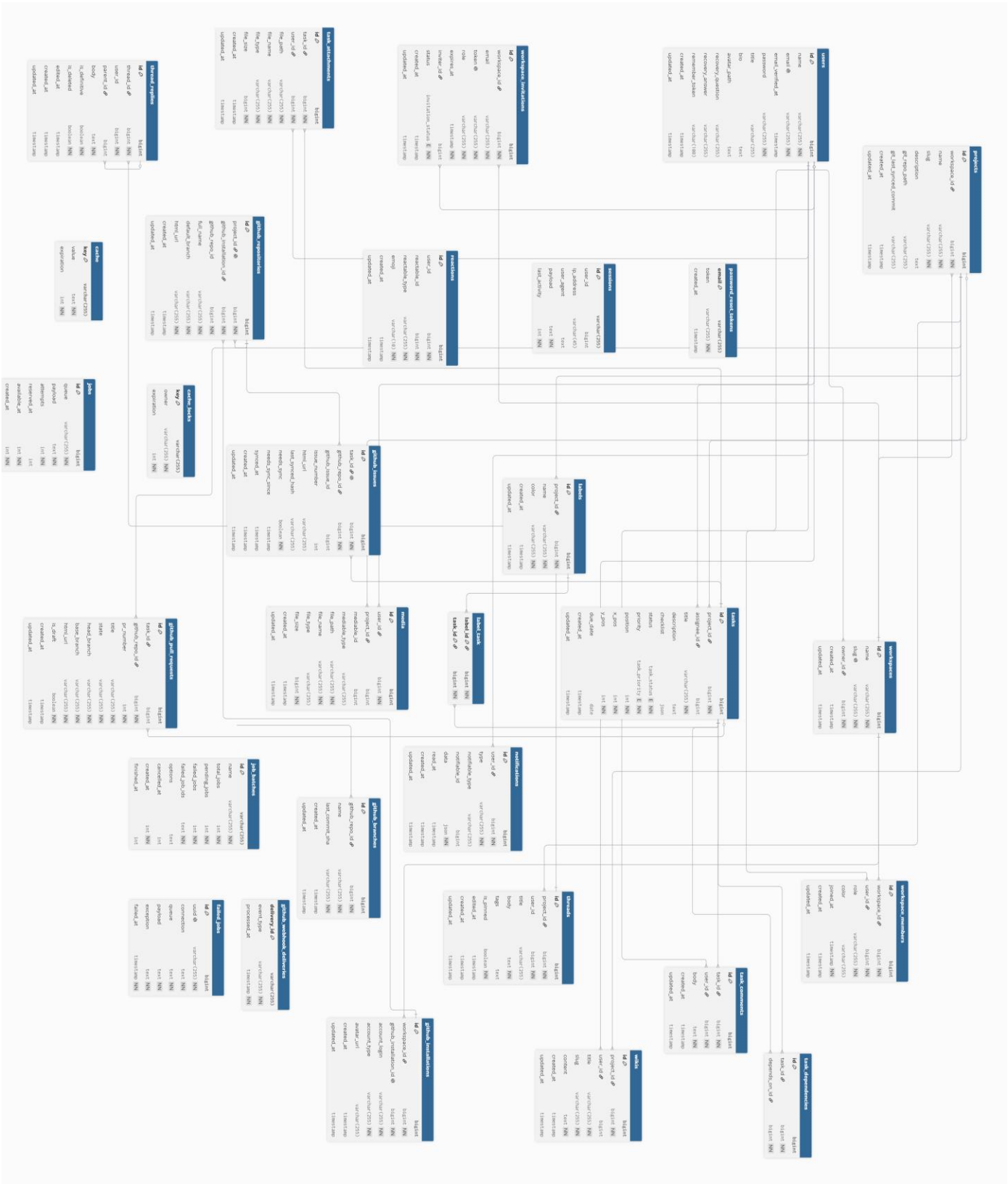
System Data Model

Entity–Relationship Diagram

The data model is centered on **users**, **workspaces**, **projects**, and **tasks**. Users join workspaces through a membership pivot; each workspace contains projects; and each project contains tasks, threads, and wiki pages. Supporting tables provide labels, dependencies, replies, reactions, media, invitations, notifications, GitHub installations, repository links, webhook deliveries, and synchronization metadata.



Database Schema Diagram

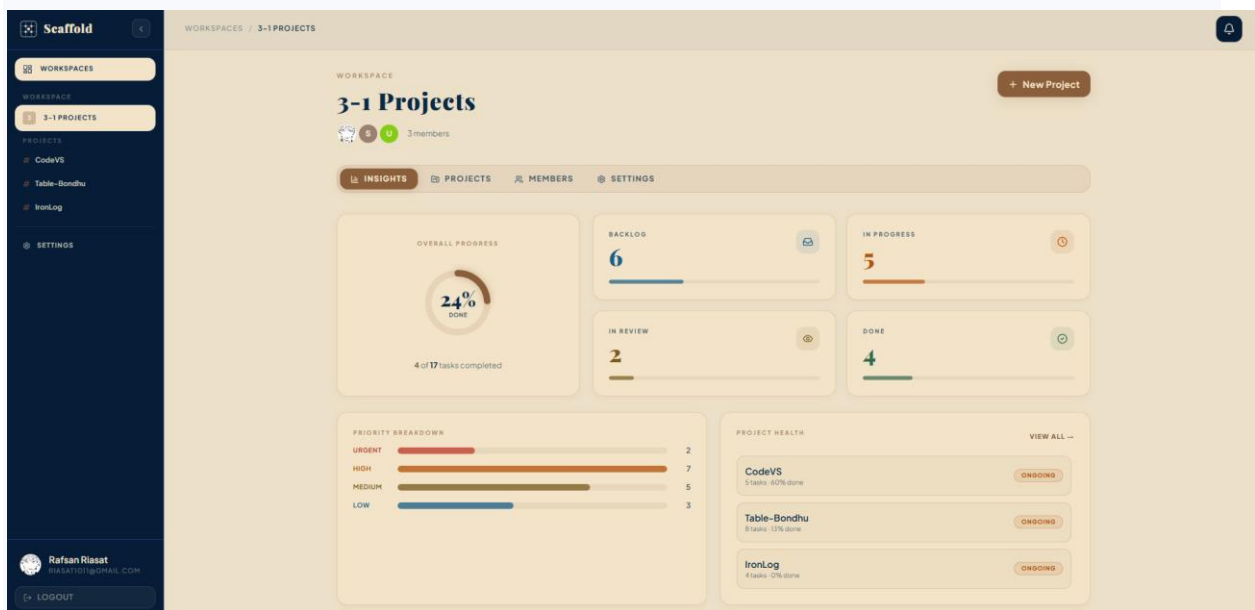


Important database behavior includes foreign-key constraints, cascading deletion where child data should not survive its owner, pivot tables for many-to-many relations, self-references for reply trees and task dependencies, polymorphic associations for reactions and media, and unique constraints for webhook deduplication. Migrations are located in `database/migrations/`, and model relationships are defined in `app/Models/`.

Technology Stack and Architecture

Feature overview

The application uses an integrated full-stack architecture rather than operating the frontend and backend as unrelated systems.



Layer	Technology	Responsibility
Backend	PHP 8.2 and Laravel 11	Routes, controllers, validation, ORM, jobs, events, security
Frontend	React 19	Interactive pages, dialogs, boards, forms, trees, filtering
Bridge	Inertia.js	Transfers controller responses and props directly to React pages
Styling	Tailwind CSS 4	Responsive utility-based design and component styling

Build tooling	Vite 5	JSX transformation, development server, optimized production assets
Database	Neon PostgreSQL	Persistent relational application data
Real time	Laravel Reverb, Echo, Pusher-JS	WebSocket broadcasting and subscriptions
External integration	GitHub Apps API	Repository linking, webhooks, installation authentication, issue sync
Hosting	Render and Docker	Hosted Apache application, workers, scheduler, and Reverb process

Authentication and Account Recovery

User capabilities

Users can register an account, log in, log out, remain authenticated through a session, and recover access through a security-question workflow. Protected routes redirect unauthenticated visitors to login, while authenticated users are redirected away from guest-only pages.

Implementation

Authentication is implemented with Laravel's session-based authentication facilities. Credentials are validated on the server, passwords are hashed, and the authenticated user's identifier is retained in the session. The account-recovery process is divided into stages: identify the account by email, answer the stored recovery question, and set a new password after verification. Recovery answers are normalized before hashing to avoid mismatches caused only by capitalization or repeated spaces. Rate limiting protects recovery endpoints from repeated attempts.

The application does not depend on a separate frontend authentication token flow. Inertia requests use the same server session and CSRF protections as ordinary Laravel forms.

Relevant files: authentication and recovery controllers under `app/Http/Controllers/`; authentication routes in `routes/web.php`; `app/Models/User.php`; pages under `resources/js/pages/Auth/`;

 Scaffold

PROJECT MANAGEMENT · REIMAGINED

Build. Ship. Together.

A minimal workspace for teams who ship.
Organize projects, track tasks, and collaborate
— without the noise.

KANBAN BOARDS

TASK TRACKING

WIKIS

THREADS

GIT ACTIVITY

Welcome back

Sign in to continue to your workspace.

EMAIL ADDRESS

PASSWORD



☐ REMEMBER ME

Forgot password?

Sign In

OR

New to Scaffold? [Create an account](#) →

 Scaffold

JOIN THE WORKSPACE

Your team is waiting for you.

Get started in seconds. No credit card, no noise
— just a clean workspace built for focused
teams.

Create your account

Join Scaffold and start building with your team.

FULL NAME

EMAIL ADDRESS

JOB TITLE

SHORT BIO

PROFILE PHOTO

PHOTO

CHOOSE PHOTO

OR PASTE A PROFILE IMAGE URL

PASSWORD



CONFIRM PASSWORD



 **OPTIONAL ACCOUNT RECOVERY**
Use a private question whose answer is memorable but
difficult for others to discover.

RECOVERY QUESTION

RECOVERY ANSWER

Middleware, Authorization, and Request Security

Feature overview

Middleware provides cross-cutting request protection. The system uses Laravel's web middleware stack for sessions, cookies, CSRF validation, route-model binding, authentication, guest restrictions, and throttling. Inertia middleware shares common properties—such as the current user—with React pages.

Implementation

- **Authentication middleware** restricts workspace, project, profile, and collaboration routes to logged-in users.
- **Guest middleware** protects login and registration screens from authenticated users.
- **CSRF middleware** rejects forged state-changing browser requests.
- **Throttle middleware** limits sensitive flows, particularly password recovery.
- **Route-model binding** converts URL identifiers into model instances and returns appropriate errors for missing records.
- **Inertia middleware** defines shared frontend props and versioning behavior.
- **Controller-level authorization** verifies workspace membership, ownership, or project association before data is read or modified.
- **Broadcast-channel authorization** confirms that a user belongs to the workspace associated with a real-time channel.

This layered approach means URL access, form submission, and WebSocket subscription are each authorized rather than relying only on hidden frontend controls.

Relevant files: `bootstrap/app.php`, `app/Http/Middleware/HandleInertiaRequests.php`, `routes/web.php`, `routes/channels.php`, and authorization checks within controllers.

Workspace Management

User capabilities

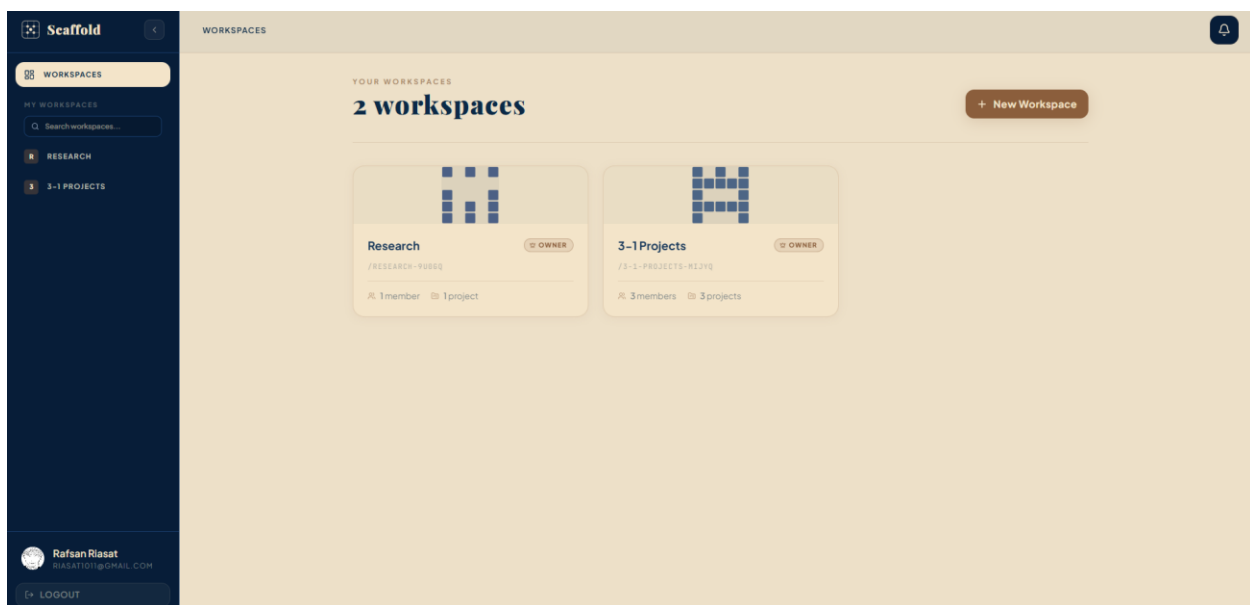
A user can create a workspace, view workspaces to which they belong, invite collaborators, manage membership, and delete a workspace when authorized. Each membership can store workspace-specific metadata such as a display color.

Implementation

Workspace is the primary data-isolation boundary. A many-to-many relationship connects users and workspaces through a membership pivot. Controllers scope workspace operations to authenticated members and perform owner checks for destructive actions. Workspace slugs provide readable URLs. Invitations are represented separately so a user can accept or decline access through a controlled token flow.

Deleting a workspace removes related projects and their dependent records according to database foreign-key rules. This prevents orphaned tasks, discussions, and repository links.

Relevant files: `app/Models/Workspace.php`, workspace and invitation controllers, workspace-related migrations, `resources/js/pages/Workspace/`, `resources/js/layouts/WorkspaceLayout.jsx`, `tests/Feature/WorkspaceTest.php`.



Project Management

User capabilities

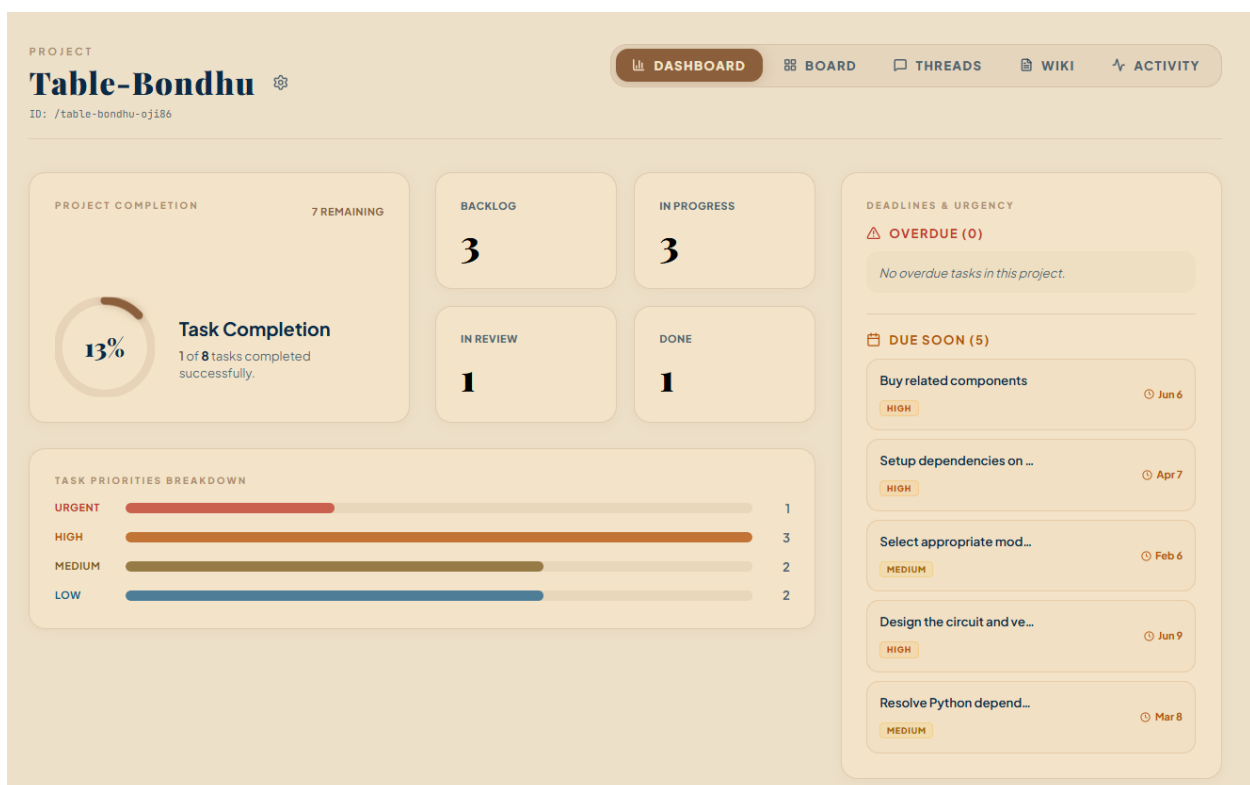
Workspace members can create projects, open a project dashboard, update project information, and delete projects subject to authorization. Projects group tasks, labels, threads, wiki pages, activity, and GitHub settings under a single workspace.

Implementation

A project belongs to one workspace, so access is inherited from workspace membership. Project controllers load related records and return Inertia pages with the data needed by project dashboards and boards. Relationships are defined through Eloquent models and reinforced by foreign keys. Project-level route structures make it difficult to mix records from unrelated workspaces accidentally.

The project module acts as the functional hub of the platform: most feature routes are nested around a project, and broadcast authorization resolves project ownership before allowing subscriptions.

Relevant files: `app/Models/Project.php`, `app/Http/Controllers/ProjectController.php`, project migrations, pages under `resources/js/pages/Project/`, and `routes/web.php`.



Task Management and Kanban Board

User capabilities

Users can create, edit, assign, label, prioritize, schedule, and delete tasks. Tasks move among statuses such as backlog, in progress, in review, and done. The Kanban board groups tasks visually by status and enables rapid progress tracking.

Implementation

Task input is validated by Laravel before persistence. Eloquent relationships connect tasks to projects, assignees, labels, comments, dependencies, and GitHub synchronization metadata. Status updates trigger safeguards and can be broadcast to other connected clients. The React task interface maintains modal, filtering, selection, and board-interaction state, while Inertia refreshes canonical data from the server.

Labels use a many-to-many pivot and can be attached or detached without duplicating task records. Due dates and selected attributes are cast into appropriate PHP types by the model.

Relevant files: `app/Models/Task.php`, `app/Http/Controllers/TaskController.php`, `app/Observers/TaskObserver.php`, task-related migrations, task pages/components under `resources/js/`, `tests/Feature/TaskTest.php`, `tests/Feature/LabelTest.php`.

The screenshot displays the 'Table-Bondhu' project management interface. The top navigation bar includes links for DASHBOARD, BOARD (active), THREADS, WIKI, and ACTIVITY. The main workspace is a Kanban board with four columns: BACKLOG (3 items), IN PROGRESS (3 items), IN REVIEW (1 item), and DONE (1 item). Each column contains task cards with titles, due dates, and IDs. For example, in the 'IN PROGRESS' column, tasks include 'Setup dependencies on Arduino IDE' (due APR 7, #16), 'Select appropriate models to choose' (due FEB 6, #15), and 'Resolve Python dependencies' (due MAR 8, #18). A 'DELETE' button is visible in the bottom right corner of the board area.

Below the Kanban board, the 'ACTIVE WORKLOAD & TASK ASSIGNMENTS' section lists team members and their task counts across different stages:

Member	Backlog	Active	Review	Done	Total Tasks
Rafsan Riasat (riasat1011@gmail.com)	2	0	1	0	3
Sabbir Bhai (sabbir@mail.com)	1	1	0	1	3
Utsa Roy (utsa@mail.com)	0	2	0	0	2

The 'RESOURCE ALLOCATION' section on the right shows 'Unassigned Members' with a note: 'Workspace members with 0 tasks assigned in this project. Use this list to find available resources.' It also states 'All workspace members are active in this project.' and shows a 'TOTAL TEAM SIZE' of 3 MEMBERS.

SCAFFOLD NUCLEUS
REF # 4

R

YOU ARE EDITING

Project Initialization

IN PROGRESS

HIGH

mm/dd/yyyy

RAFSAN RIASAT

GITHUB SYNC

Create & Link GitHub Issue

DESCRIPTION

WRITE

PREVIEW

B I U H1 H2 H3

Ctrl/Cmd+B, I, U Ctrl/Cmd+Alt+1/2/3

Need to initialize the project which will be based on Laravel using React as the front end and Inertia as the bridge. It will be using Oracle XE21 as the database and must have PLSQL functionalities

PROJECT STREAM

S SABBIR BHAI 17:50

tumakei kaj korte hobe bhaiya

RAFSAN RIASAT 17:50

kaj kemon cholche?

RAFSAN RIASAT 17:50

hi bro

Broadcast a message...

LINK WIKI

CHECKLIST

0/3 Complete

Initialize the seeders

Create the migrations

Seed data

+ Enter a new checkpoint...

BLOCKERS

+ ADD BLOCKER

#4 Project Initialization

#3 Setup Oracle Database

Task Dependencies and Flow View

User capabilities

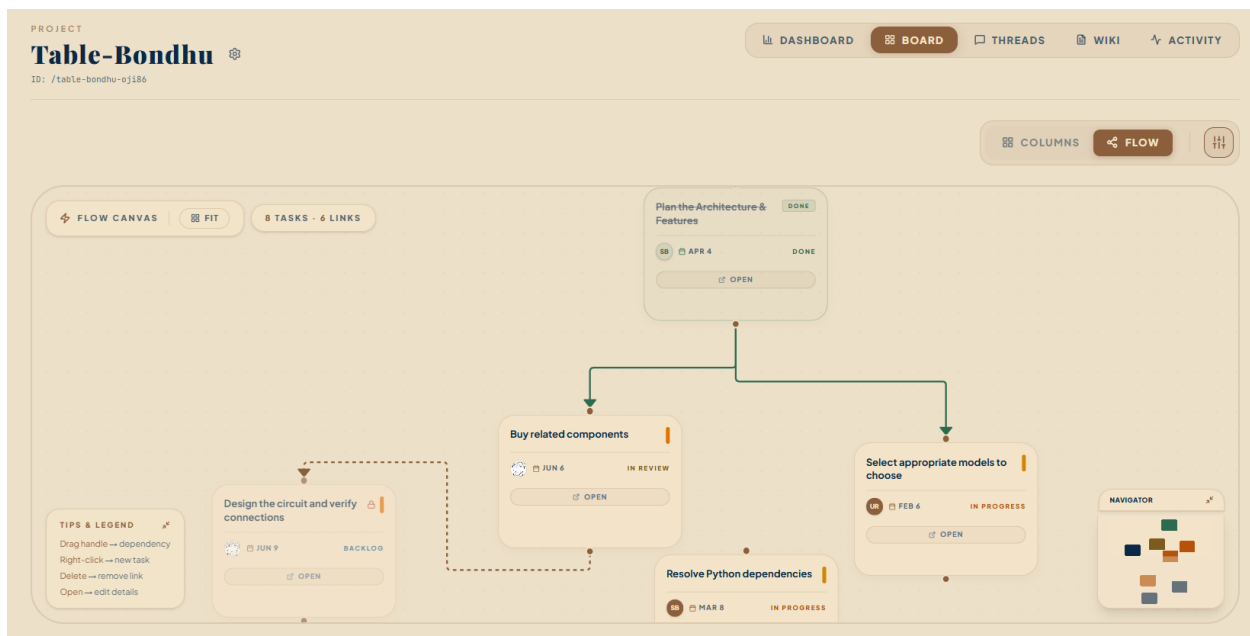
Tasks can depend on other tasks. Users can inspect these relationships in a flow-oriented view and understand which unfinished prerequisite blocks a task. Invalid circular relationships are rejected.

Implementation

Dependencies are stored in a self-referencing many-to-many table. Both the frontend and backend implement cycle checks. The server performs a breadth-first traversal of dependency links before accepting a new edge, ensuring that adding the relationship cannot create a path back to the original task. The frontend mirrors this behavior for immediate feedback, but the server remains authoritative.

When a previously completed prerequisite is moved away from **done**, dependent tasks that are no longer validly complete can be reverted recursively. This preserves workflow consistency rather than displaying completed work whose prerequisites are unfinished.

Relevant files: `app/Http/Controllers/TaskController.php`, `resources/js/utils/cycleDetection.js`, `resources/js/utils/taskDependencies.js`, task dependency migrations, flow-view components, `tests/Feature/TaskTest.php`.



Threads, Replies, and Reactions

User capabilities

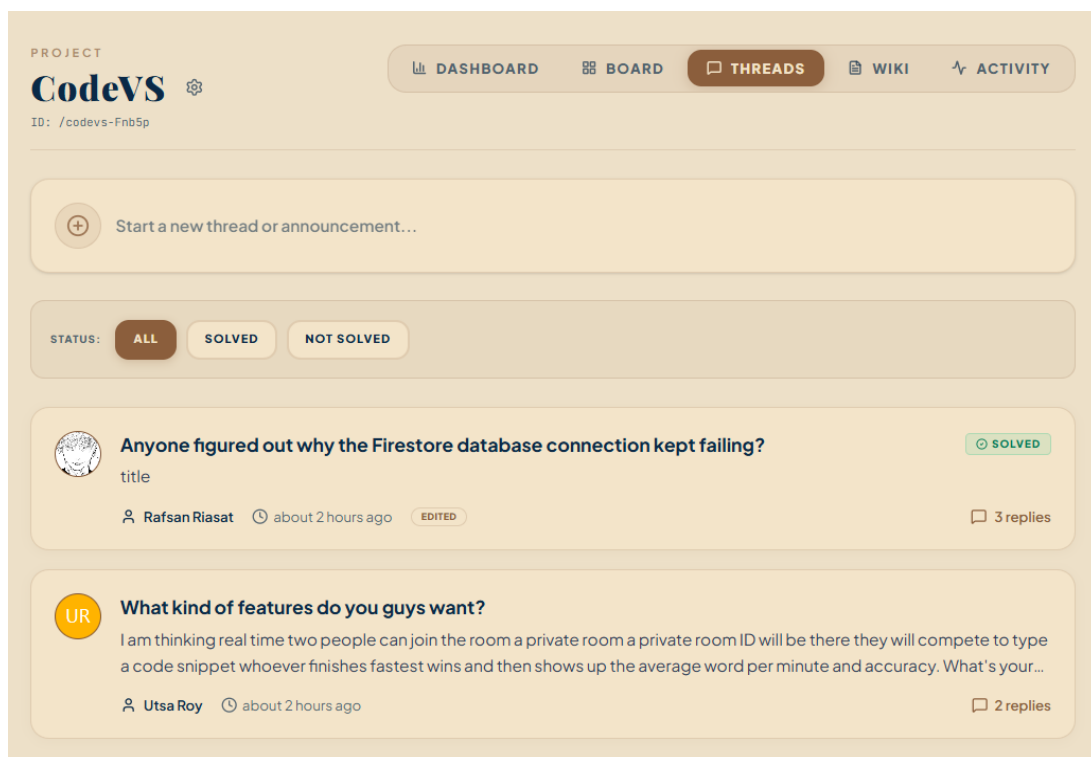
Project members can create discussion threads, reply to threads, reply to other replies, react to supported content, and mark a suitable reply as the definitive answer. This supports question-and-answer and design-discussion workflows inside a project.

Implementation

Thread replies use a self-referencing `parent_id`, allowing a reply tree of arbitrary depth. React recursively renders this hierarchy through a comment-tree component. A definitive-answer reference identifies the accepted response. Reactions use a polymorphic relationship so one reaction implementation can support threads, replies, task comments, and tasks.

Presence-aware subscriptions provide information about users participating in the discussion, while broadcasts propagate changes without requiring a full refresh.

Relevant files: `app/Models/Thread.php`, `ThreadReply.php`, and `Reaction.php`; `thread`, `reply`, and `reaction` controllers; thread-related migrations; pages and recursive components under `resources/js/`; `tests/Feature/ThreadTest.php`.





Wiki and Project Documentation

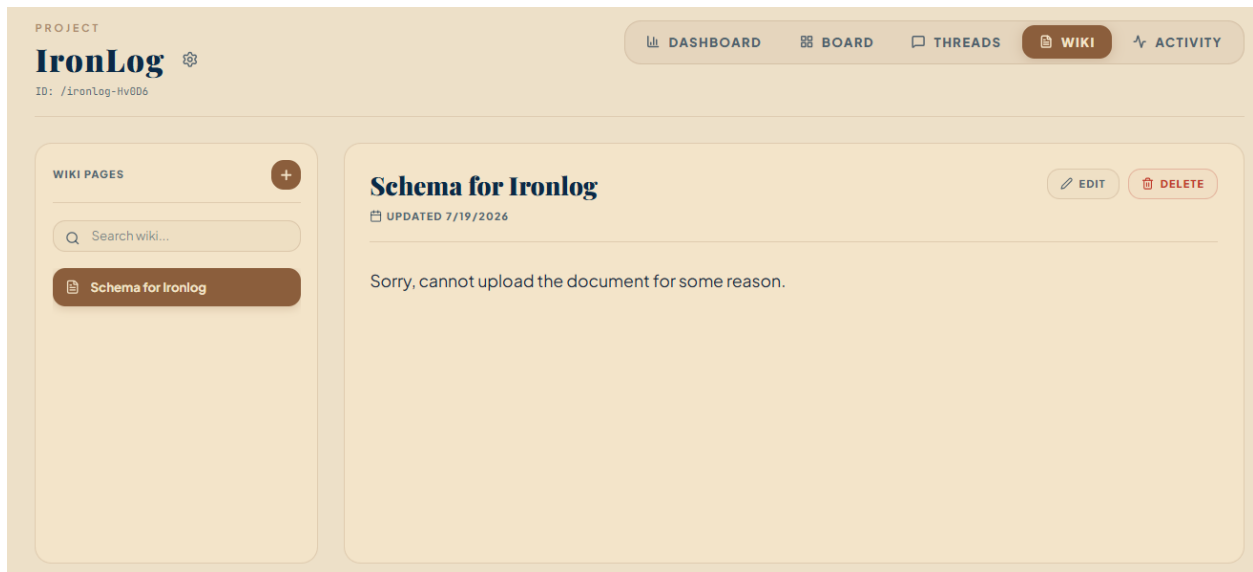
User capabilities

Users can create, edit, view, and remove wiki pages associated with a project. Wiki pages provide a lightweight place for requirements, setup notes, decisions, or other persistent documentation.

Implementation summary

Wiki content is stored relationally and linked to both its project and author. Titles generate readable slugs; updates can regenerate the slug when the title changes. Laravel controllers validate and persist content, while React pages provide editing and display interfaces. Markdown-oriented content can be presented as formatted documentation rather than plain input text.

Relevant files: `app/Models/Wiki.php`, wiki controller, wiki migrations, wiki pages/components under `resources/js/`, `tests/Feature/WikiTest.php`.



GitHub App and API Integration

User capabilities

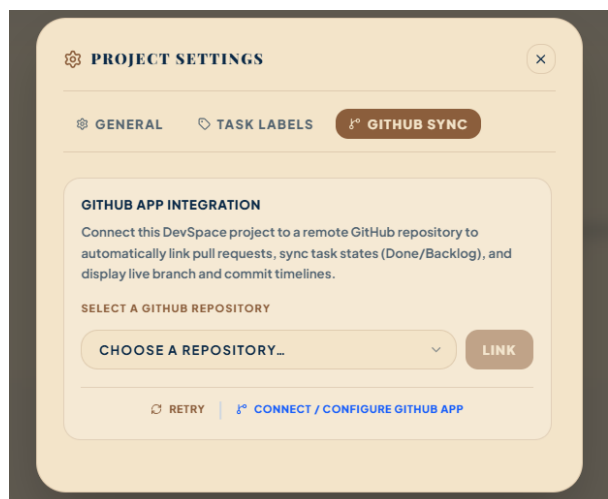
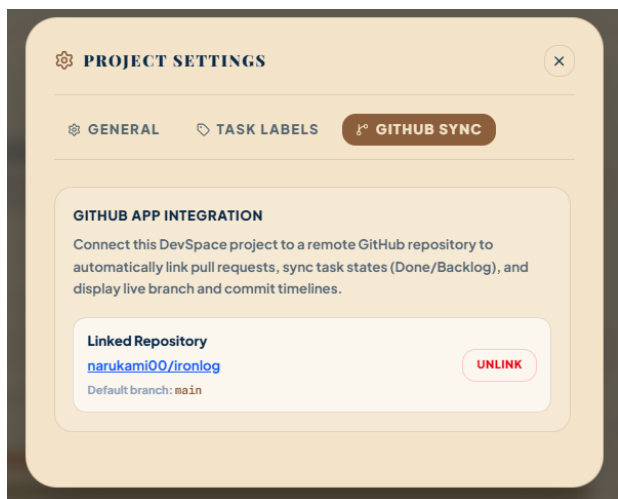
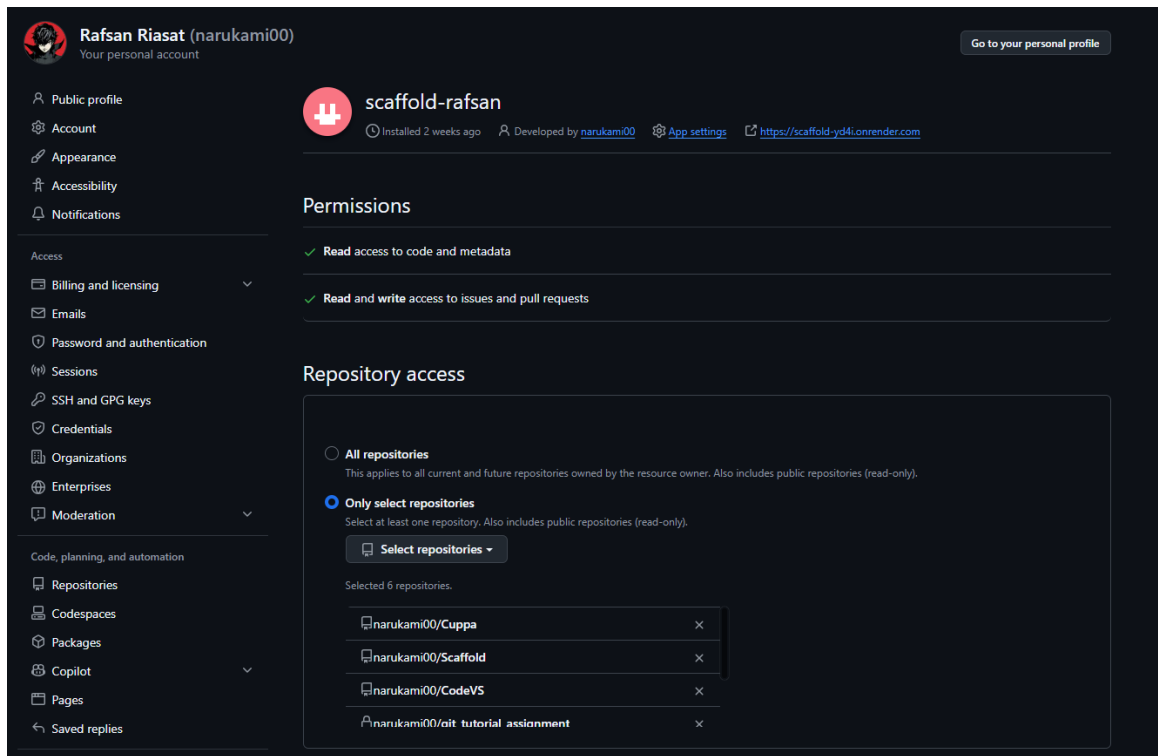
A workspace can connect a GitHub App installation, inspect repositories available to that installation, link a project to a repository, and synchronize suitable project-task activity with GitHub issues. Repository activity can also appear inside Scaffold.

Implementation

This integration uses the GitHub Apps API rather than a personal access token. The server creates a short-lived RS256 JWT from the App private key and exchanges it for an installation access token. Installation tokens are cached for approximately 55 minutes to reduce repeated authentication requests. API calls are made from the Laravel backend so private credentials are never exposed to the browser.

Projects store repository-link information, while a dedicated service creates or updates GitHub issues. A content hash records the last synchronized title, description, and state; this acts as an echo-loop guard when the same change returns through a webhook. Marker content can associate an issue with its Scaffold task.

Relevant files: GitHub integration controller, `app/Services/GitHubTokenService.php`, GitHub synchronization service, GitHub models and migrations, `config/services.php`, GitHub pages/components, `tests/Feature/GitHubIntegrationTest.php`.



Webhook API and Bidirectional Synchronization

Feature overview

GitHub sends webhook HTTP requests when repository activity changes. Scaffold receives these requests, verifies their authenticity, prevents duplicate processing, and dispatches background work that updates local state.

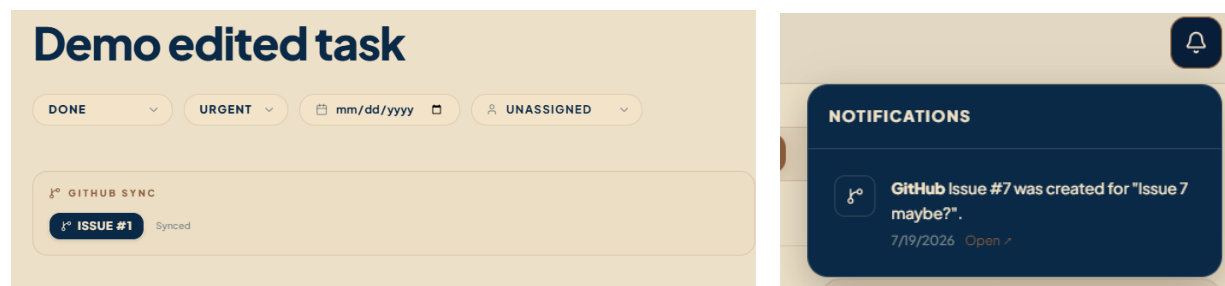
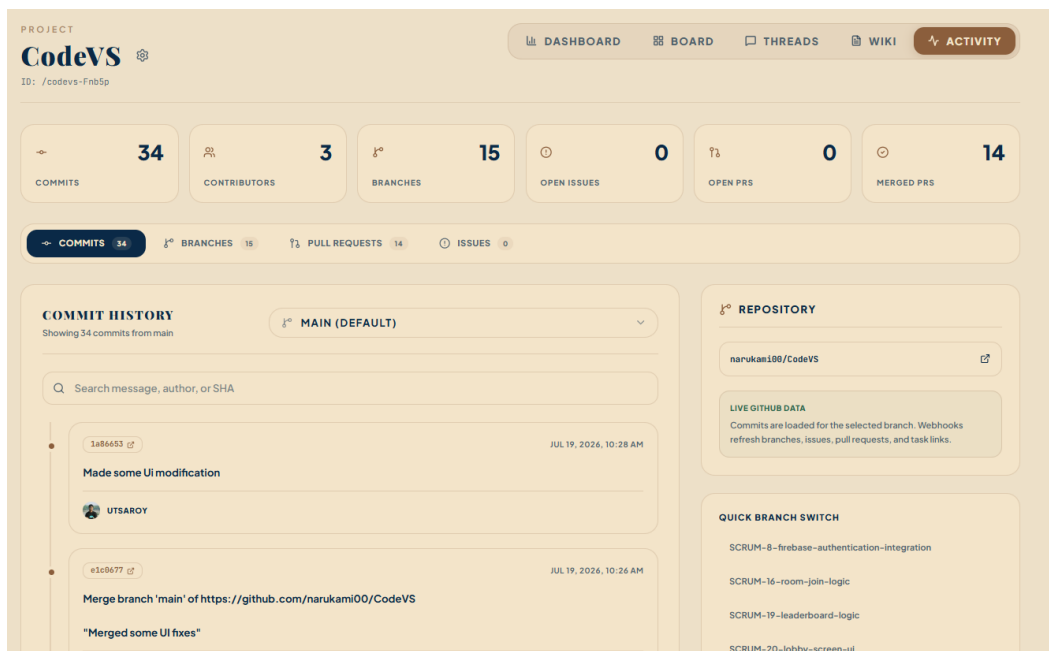
Implementation

The webhook endpoint verifies the `X-Hub-Signature-256` header using an HMAC SHA-256 digest of the raw request body and the configured webhook secret. Requests with missing or invalid GitHub headers are rejected. The `X-GitHub-Delivery` identifier is stored with a uniqueness guarantee so a redelivered webhook is not processed twice.

Valid payloads dispatch `ProcessGitHubWebhookJob`. The job interprets supported event types—such as issues, pushes, commits, branches, and pull requests—and updates tasks or activity data. Outbound synchronization is handled by a separate queued job. Queue workers keep API work outside the interactive browser request where appropriate.

The application therefore exposes a focused webhook API while ordinary page navigation continues to use Inertia rather than a large public REST API.

Relevant files: webhook routes in `routes/web.php`, GitHub integration controller, `app/Jobs/ProcessGitHubWebhookJob.php`, outbound synchronization job, `webhook-delivery` migration, `tests/Feature/GitHubIntegrationTest.php`, `tests/Feature/GitSyncTest.php`.



Demo Task 2

IN REVIEW

HIGH

mm/dd/yyyy

UNASSIGNED

GITHUB SYNC

ISSUE #2 Synced

PROJECT STREAM

RAFSAN RIASAT 20:34

Commit
fac7374b2f870688c2a43bfa26fe3a77323f3d9
by Rafsan Riasat:
lets see if works #2

Demo task 3

DONE

MEDIUM

mm/dd/yyyy

UNASSIGNED

GITHUB SYNC

ISSUE #3 Synced

PROJECT STREAM

RAFSAN RIASAT 20:41

Task closed via commit
d804894f67595f8398567f23441383a1a74d75e8
by Rafsan Riasat:
closes #3

Demo Task 4

DONE

LOW

mm/dd/yyyy

UNASSIGNED

GITHUB SYNC

ISSUE #4 Synced

LINKED PULL REQUESTS

MERGED #5 fixes #4 demo-branch → main

PROJECT STREAM

RAFSAN RIASAT 20:47

Commit
9d26a9b838eb1a820abf8d48d05e01b729e8c33
by Rafsan Riasat:
Merge pull request #5 from narukami00/demo-branch
fixes #4

RAFSAN RIASAT 20:47

Pull Request #5 merged:
fixes #4

Open 3 Closed 3

Author Labels Projects Milestones Assignees Newest

☐	☒	Demo Task 4	#4 · by scaffold-rafsan was closed 2h ago
☐	☒	Demo task 3	#3 · by scaffold-rafsan was closed 2h ago
☐	☒	Demo edited task	#1 · by scaffold-rafsan was closed 3h ago

Real-Time Collaboration

User capabilities

Users can see task, thread, notification, and collaboration updates with minimal manual refresh. Presence channels can indicate who is currently viewing or interacting with shared resources.

Implementation

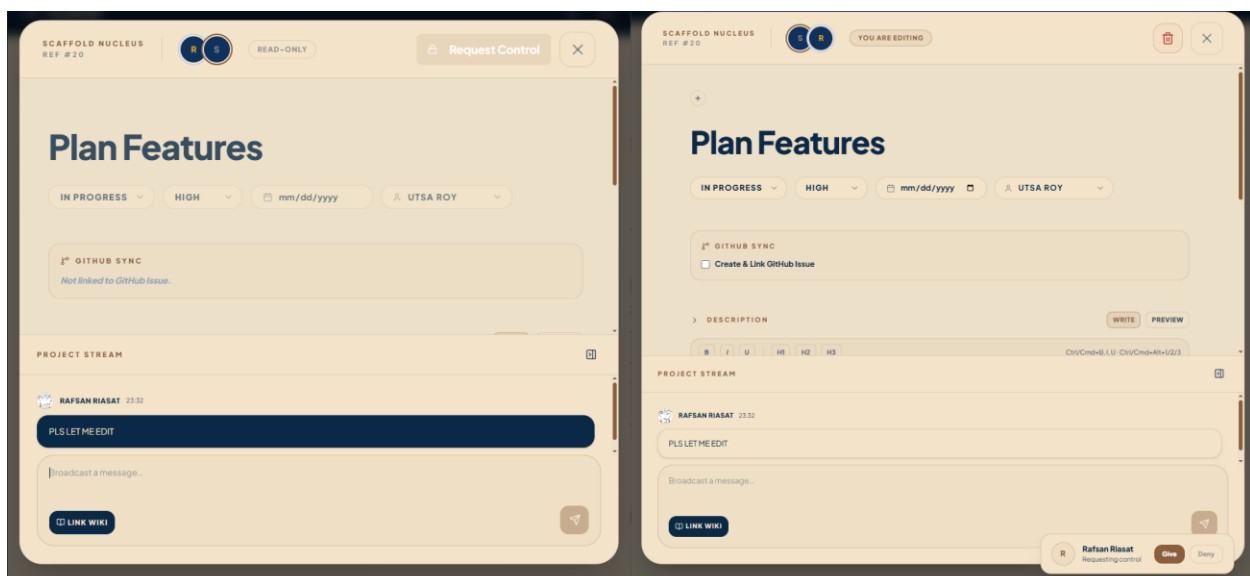
Laravel events implement synchronous broadcasting and publish structured payloads to private or presence channels. Reverb hosts the WebSocket server using the Pusher protocol. Laravel Echo and Pusher-JS subscribe from React pages. Channel authorization verifies that the authenticated user belongs to the workspace that owns the requested entity.

Typical flow:

1. A controller or observer changes a record.
2. A Laravel broadcast event is dispatched.
3. Reverb publishes the event to an authorized channel.
4. Echo receives it in the browser.
5. React updates local state or requests fresh Inertia data.

This mechanism is used for task updates, locking/control messages, thread activity, presence, and user notifications.

Relevant files: `app/Events/`, `routes/channels.php`, `config/reverb.php`, `config/broadcasting.php`, `resources/js/echo.js`, React subscription code, `docker/start.sh`.



Notifications and Activity

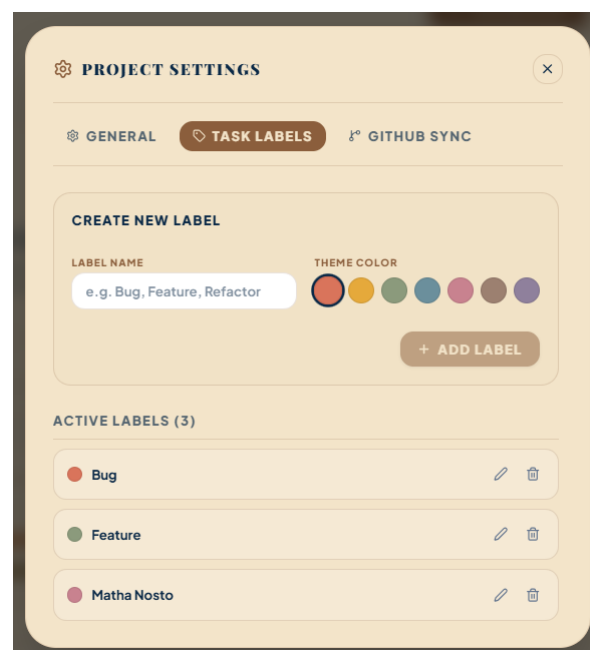
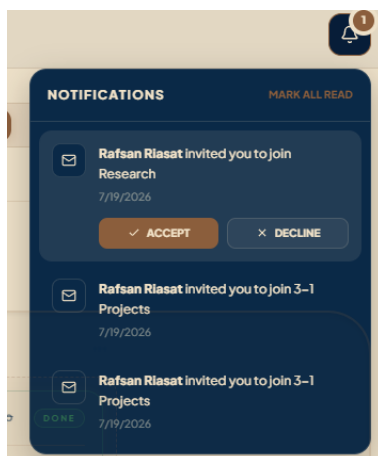
User capabilities

Users receive notifications about relevant application actions and can view or mark them as read. Project activity provides a summarized view of important events, including selected GitHub-originated activity.

Implementation summary

A notification helper centralizes record creation and event dispatch. Notifications are persisted so they survive reconnection, while a private user channel delivers immediate updates to the notification panel. Read actions update stored state and keep the user interface synchronized. Activity views aggregate records associated with the current project or authorized user.

Relevant files: `app/Helpers/Notifier.php`, `notification controller/model/event`, `notification migrations`, `resources/js/components/NotificationPanel.jsx`, `dashboard/activity` pages.



User Profiles, Avatars, and Search

User capabilities

Users can view global or workspace-specific profiles, edit their name, title, biography, color, recovery settings, and avatar, and inspect assigned-task statistics. Search allows authorized users to locate relevant application content.

Implementation summary

Profile statistics are computed from tasks, wikis, and threads scoped to the relevant workspace memberships. Avatar input supports validated remote URLs and uploaded image formats. Where the PHP GD extension is available, uploaded images are center-cropped and resampled to a 300×300 PNG. Previous locally stored avatars are deleted safely when replaced.

Search queries return application entities visible to the current user rather than exposing records across unrelated workspaces. Media support uses a polymorphic design where one attachment mechanism can be associated with multiple supported content types.

Relevant files: `app/Http/Controllers/ProfileController.php`, search and media controllers, `app/Support/AvatarUrl.php`, `app/Models/Media.php`, profile/search pages, `tests/Feature/ProfileTest.php`.

The screenshot displays a user profile for Rafsan Riasat, a student. The profile includes a circular avatar, a bio stating 'I like building stuff.', and contact information: RIASAT1011@GMAIL.COM. It also shows workspace statistics: 3-1 PROJECTS. Below the profile are five stat boxes: 2 TASKS COMPLETED, 5 TASKS PENDING, 3 TASKS BLOCKED, 1 WIKIS WRITTEN, and 2 THREADS STARTED. A section titled 'WORK ASSIGNMENTS (7)' contains a table with 5 columns: TASK, PROJECT, STATUS, PRIORITY, and DUE DATE.

TASK	PROJECT	STATUS	PRIORITY	DUE DATE
Project Initialization	IronLog	IN PROGRESS	HIGH	4/7/2027
Buy related components	Table-Bondhu	IN REVIEW	HIGH	6/6/2027
Design the circuit and verify connections	Table-Bondhu	BACKLOG	HIGH	6/9/2027
Connect Laravel with Oracle	IronLog	BACKLOG	HIGH	7/7/2027
Code up the ESP32 features	Table-Bondhu	BACKLOG	LOW	9/9/2027
Initialize React app	CodeVS	DONE	URGENT	-
Connect Firebase with React app	CodeVS	DONE	HIGH	-



FULL NAME

Rafsan Riasat

JOB TITLE

Student

SHORT BIO

I like building stuff.

PROFILE IMAGE URL

https://example.com/profile.jpg

Paste a Google "Copy image address" link (gstatic) or any direct .jpg/.png/.webp URL

 **PASSWORD RECOVERY** 

A recovery question is configured. Enter a new answer and your current password to replace it.

RECOVERY QUESTION

Enter a new question to replace the current one

NEW RECOVERY ANSWER

Enter a new answer to replace it

SIGNATURE WORKSPACE COLOR

CANCEL

SAVE PROFILE

TEAM MEMBERS

 **Rafsan Riasat** YOU
riasat101@gmail.com

OWNER

 **Sabbir Bhai**
sabbir@mail.com

MEMBER

 **Utsa Roy**
utsa@mail.com

MEMBER

PENDING INVITATIONS

 **sabbir@gmail.com**
ROLE MEMBER

DECLINED

INVITE NEW MEMBER

Members receive a notification instantly if they have a Scaffold account.

EMAIL ADDRESS

colleague@example.com

 Send Invite

JavaScript, React, and Inertia Functionality

Feature overview

The frontend is a React single-page experience delivered through Laravel and Inertia. It provides modals, navigation, filters, boards, trees, forms, live updates, and reusable interface elements without duplicating backend authorization logic.

Implementation

- **React state and effects** manage dialogs, form values, selected tasks, board filters, tree expansion, and Echo subscriptions.
- **Inertia links and router actions** navigate pages and submit mutations while preserving the server-driven routing model.
- **Inertia forms** display Laravel validation errors close to the relevant fields.
- **Reusable components** implement task cards, notification panels, comments, profile controls, project navigation, and confirmation dialogs.
- **Utility modules** provide client-side dependency traversal and task-relationship helpers.
- **Vite** transforms JSX, performs hot-module replacement during development, and creates optimized production bundles.
- **Tailwind CSS** supplies responsive layouts, states, typography, spacing, colors, and component visuals.
- **Echo subscriptions** connect selected pages and components to real-time channels and clean up listeners during unmount.

The frontend assists usability but does not replace server validation or authorization. Any operation initiated from JavaScript is rechecked by Laravel.

Relevant files: `resources/js/app.jsx`, `bootstrap.js`, `echo.js`, `pages/`, `components/`, `layouts/`, `utils/`, `vite.config.js`, `resources/css/app.css`.

Database Functionality and Data Integrity

Feature overview

The relational database is responsible not only for storage but also for preserving valid relationships and supporting efficient scoped queries.

Implementation

- Primary and foreign keys identify records and enforce ownership links.
- Pivot tables implement many-to-many workspace membership, labels, and dependencies.
- Cascading foreign keys clean up subordinate data when a parent is deleted.
- Unique indexes prevent duplicate values and webhook deliveries where required.
- Nullable foreign keys allow optional assignees or selected answers while preserving references when present.
- Model casts convert date, Boolean, and structured values into appropriate PHP representations.
- Eloquent eager loading reduces repeated queries when pages need related workspace, project, label, and user information.
- PostgreSQL-specific connection handling supports Neon/PgBouncer behavior, including emulated prepares and Boolean-binding compatibility.

Local development began with MySQL through XAMPP. Hosted deployment moved to Neon PostgreSQL because GitHub webhooks and external integration required a publicly reachable, persistent application environment. The migration layer allows the same domain model to be recreated consistently across environments.

Relevant files: `database/migrations/`, `app/Models/`, `config/database.php`, `app/Database/PostgresConnection.php`, `app/Providers/AppServiceProvider.php`.

Deployment

The hosted demonstration runs in a Docker container on Render. Apache serves Laravel and static assets, while the startup script also runs the queue worker, scheduler, and Reverb server. Apache WebSocket proxy modules forward WebSocket traffic to Reverb. Persistent application data is stored in Neon PostgreSQL rather than inside the disposable container filesystem.

Required secrets—database URL, application key, GitHub App identifiers, private key, webhook secret, and Reverb credentials—are supplied through environment variables and are not embedded in frontend code.

Relevant files: `Dockerfile`, `docker/start.sh`, `docker/000-default.conf`, `.env.example`, Laravel configuration files.

Project dashboard

Connect

Import data

Share

Get connected to your new database

Connection string

Copy your project's connection string and add it to your app config.

> _ Neon init

Set up your local dev environment with a single command.

IDE extension

Set up your app with the Neon extension in VS Code or Cursor.

MCP server

Connect your app to Neon's MCP server with supported AI tools.

Dismiss

Compute

1.1 / 100 CU-hrs

Storage

0.03 / 0.5 GB

History

0 GB

Network transfer

0 / 5 GB

Usage since Jul 8, 2026. Metrics may be delayed by an hour and are not updated for inactive projects. [Learn more.](#)

Monitoring

View all metrics

Branch

production (Default)

Compute

Primary (Active)

Refresh

RAM (GB)

0.25

2

1 / 10 Branch

View all

Name

Primary compute

Created by

% production (Default)

.25 ↔ 2 CU (Active)

Preview Workflow

X

You don't have any preview branches yet. Improve your workflow by adding database branching to your development previews.

Install an integration

WEB SERVICE

Scaffold

Docker

Free

Upgrade your instance →

Connect

Manual Deploy

Service ID: `srv-d9764rpo3l8c73a2qfs0`

`narukami00 / Scaffold` `main`

`https://scaffold-yd4i.onrender.com`

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. [Upgrade now](#)

Filter events

31

Deploy live for `9017a82`: feat: project deletion added

July 19, 2026 at 10:43 PM

Deploy started for `9017a82`: feat: project deletion added

New commit via Auto-Deploy

July 19, 2026 at 10:42 PM

Deploy live for `c87b2d2`: fix: profile loading fixed — a small layout wrapper with usePage() to pick AppLayout vs WorkspaceLayout

Scaffold

Public

Pin

Watch 0

main

7 Branches

0 Tags

Go to file

Add file

Code

narukami00

feat: project deletion added

9017a82 · 1 hour ago

82 Commits

app

feat: project deletion added

1 hour ago

bootstrap

feat(github): build secure webhook receiver with HMAC-SHA...

2 weeks ago

config

fix: avoid Neon cached-plan errors after schema migrations

4 hours ago

database

feat: project deletion added

1 hour ago

docker

feat: expand GitHub activity with branches issues pull request...

3 hours ago

public

feat: project deletion added

1 hour ago

Conclusion

Scaffold demonstrates a complete transition from a local laboratory project to a hosted, multi-user collaboration platform. Its main achievement is the integration of conventional project-management features—workspaces, projects, tasks, discussions, wikis, profiles, and notifications—with technically significant capabilities such as dependency cycle prevention, recursive workflow correction, real-time broadcasting, authenticated GitHub App access, signed webhooks, event deduplication, background synchronization, and cloud deployment.

The architecture assigns clear responsibilities to each technology. Laravel controls security, validation, persistence, integrations, and events; Inertia connects server routes to client pages; React supplies a responsive interactive experience; PostgreSQL enforces durable relational integrity; and Reverb provides real-time communication. Docker and Render package these capabilities into a publicly accessible demonstration environment, while Neon provides managed database persistence.

From a laboratory perspective, the project illustrates full-stack application development across database design, backend programming, middleware, frontend component design, JavaScript interaction, external API consumption, webhook processing, WebSockets, testing, and deployment. The system is functional and extensible, although frontend automation, broader broadcast testing, and additional security and performance hardening would be valuable future improvements.

Github repository: <https://github.com/narukami00/Scaffold>

Website: <https://scaffold-yd4i.onrender.com/>